

Lectia: Conversii de tip in C++

1. Definitie si rol

Conversia de tip reprezinta transformarea unei valori dintr-un tip de date intr-un alt tip de date. Conversiile sunt esentiale in evaluarea expresiilor, in apelul functiilor si in atribuirii. In C++, conversiile pot fi automate (implicite) sau realizate explicit de programator.

2. Conversii implicite (promovari)

Conversiile implicite sunt realizate automat de compilator atunci cand este necesar. Ele apar frecvent in expresii mixte, in atribuirii sau la apelul functiilor. Scopul promovarii este obtinerea unui tip comun pentru evaluarea corecta a expresiilor.

3. Promovari aritmetice uzuale

In expresii aritmetice, tipurile de rang inferior sunt promovate la tipuri de rang superior. De exemplu, `char` si `short` sunt promovate la `int`. Daca unul dintre operanzi este de tip `real`, ceilalti sunt convertiti la tip `real`. Daca unul dintre operanzi este `double`, rezultatul expresiei va fi `double`.

4. Conversii in atribuirii

In cazul unei atribuirii, expresia din partea dreapta este convertita la tipul variabilei din stanga. Conversia poate duce la pierdere de informatie, de exemplu la atribuirea unei valori reale unei variabile `int`. Conversiile implicite in atribuirii nu genereaza eroare de compilare, dar pot produce rezultate nedorite.

5. Conversii intre tipuri integrale

Conversiile intre tipuri integrale pot modifica semnul sau intervalul valorii. Conversia de la un tip mai mare la unul mai mic poate duce la overflow. Conversiile intre `signed` si `unsigned` pot produce rezultate surprinzatoare, in special in comparatii.

6. Conversii intre tipuri reale si integrale

Conversia de la tip `real` la tip integral presupune eliminarea partii fractionare. Conversia de la tip integral la tip `real` este sigura din punct de vedere al intervalului, dar poate introduce erori de reprezentare din cauza preciziei finite a numerelor reale.

7. Conversii explicite (cast)

Conversiile explicite sunt realizate la cererea programatorului. In C++ se pot folosi conversii de forma `(tip)valoare` sau conversii moderne precum `static_cast(valoare)`. Conversiile explicite indica intentia clara a programatorului si fac codul mai usor de analizat.

8. `static_cast` si conversii sigure

`static_cast` este forma recomandata de conversie explicita in C++. Aceasta permite conversii intre tipuri compatibile si este verificata la compilare. Utilizarea `static_cast`

imbunatateste lizibilitatea si siguranta codului.

9. Conversii in expresii conditionale

In expresii conditionale, valorile sunt convertite la tip bool. Valoarea 0 este considerata false, iar orice valoare diferita de 0 este considerata true. Aceasta regula se aplica atat tipurilor integrale, cat si tipurilor reale.

10. Conversii si operatorul conditional

Operatorul conditional ? : presupune conversia celor doua expresii posibile la un tip comun. Tipul rezultatului este determinat de regulile de conversie ale limbajului si poate influenta evaluarea ulterioara.

11. Conversii si functii

La apelul unei functii, argumentele sunt convertite la tipurile parametrilor formali. Conversiile implicite pot duce la pierdere de informatie daca tipurile nu sunt alese corect. Pentru evitarea ambiguitatilor este recomandata folosirea tipurilor corecte si a conversiilor explicite.

12. Efecte secundare si conversii nedorite

Conversiile implicite pot masca erori logice si pot produce efecte secundare greu de depistat. Comparatiile intre tipuri diferite, in special intre signed si unsigned, sunt o sursa frecventa de erori.

13. Observatii importante

Conversiile trebuie intelese in contextul expresiilor complete, nu izolat. Ordinea evaluarii si regulile de promovare influenteaza rezultatul final al expresiilor.

14. Greseli frecvente

- pierdere de informatie la conversii implicite;
- comparatii intre signed si unsigned;
- utilizarea excesiva a conversiilor implicite;
- presupunerea ca conversiile reale-intreg sunt exacte.

15. Checklist de BAC

- stii diferenta dintre conversii implicite si explicite;
- intelegi regulile de promovare aritmetica;
- poti identifica pierderea de informatie;
- stii cand este necesar un cast explicit.